

SPARK technology

Based on <http://www.metz.supelec.fr/~vialle> slides

IBM certification slides

Dataflair tutorial

SPARK technology

1.Main objectives, features and components

2.Simple Spark application execution scheme

3.RDD: Resilient Distributed Dataset

4.Detailed Spark application execution scheme

5.From RDD to Dataset and Dataframe

6.Spark Streaming

1. Spark main objectives

1. Address the limitations of older data processing frameworks (such as Hadoop MapReduce) by providing a faster, simpler, and more versatile computing engine.
2. Offer a “unified” engine capable of handling multiple types of workloads within a single platform.
 - Distributed engine for Big data
 - Enables batch and streaming processing, Machine Learning capabilities, Graph processing
 - High-level APIs (DataFrame/SQL)

Apache Spark Architecture is based on two main abstractions:

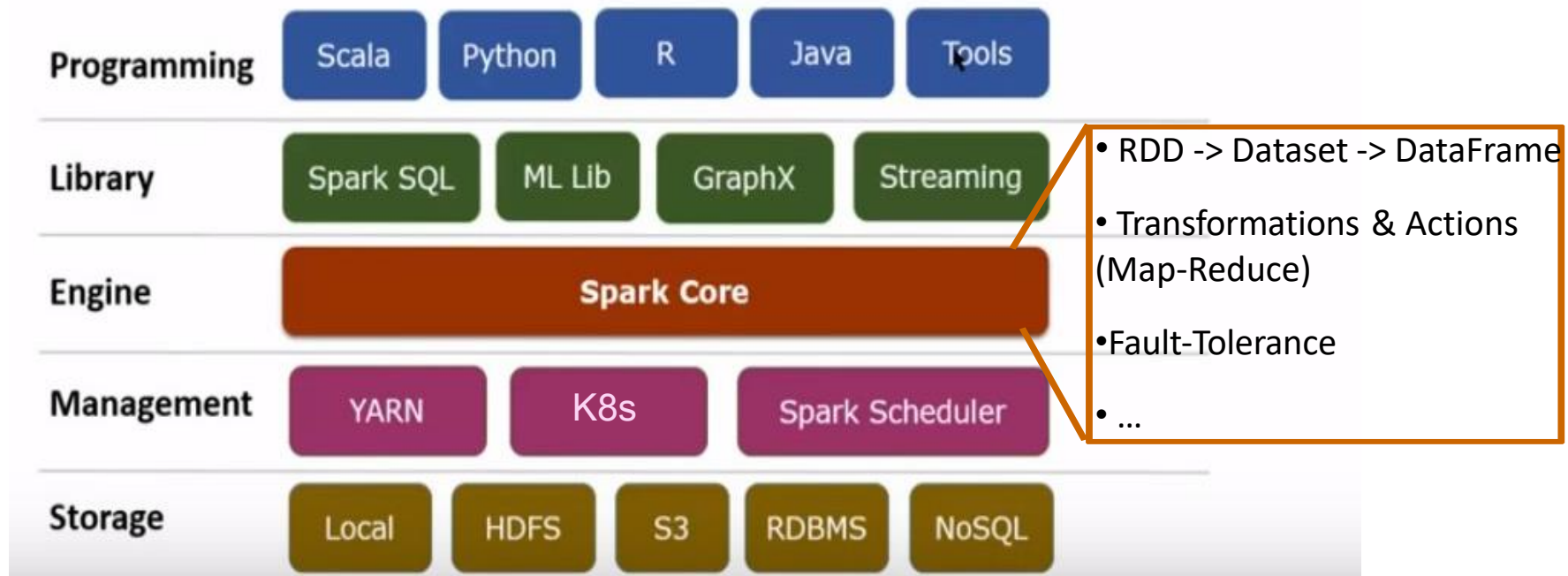
- *Resilient Distributed Dataset (RDD)*
- *Directed Acyclic Graph (DAG)*

1. Spark main features



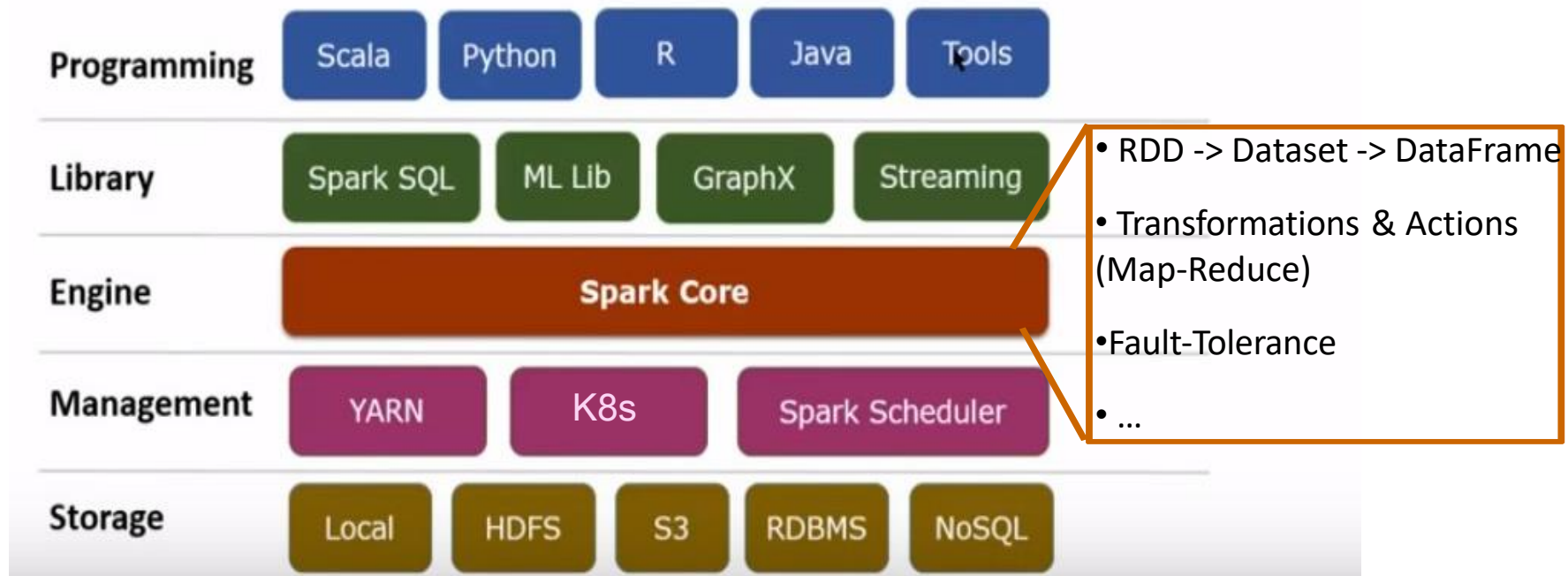
- 1. Speed** up to 100 times faster than Hadoop MapReduce for large-scale data processing. It is also able to achieve this speed through controlled partitioning and In-memory treatment.
- 2. Powerful Caching** Simple programming layer provides powerful caching and disk persistence capabilities.
- 3. Deployment** through Mesos (old), K8s, Hadoop via YARN, or Spark's own cluster manager.
- 4. Real-Time** Real-time computation & low latency because of in-memory computation.
- 5. Polyglot** Provides high-level APIs in Java, Scala, Python, and R. Spark code can be written in any of these four languages. It also provides a shell in Scala and Python.

1. Spark components



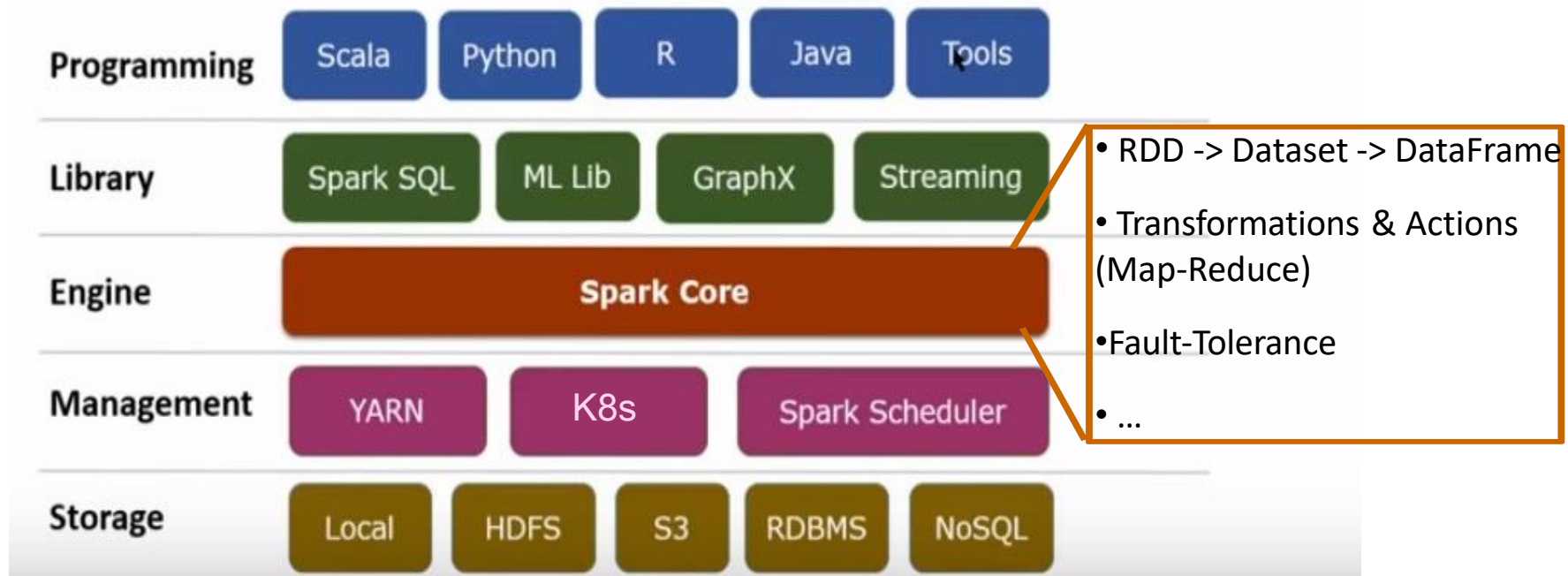
1. Spark Core is a central point of Spark. Basically, it provides an execution platform for all the Spark applications. Moreover, to support a wide array of applications, Spark Provides a generalized platform..

1. Spark components



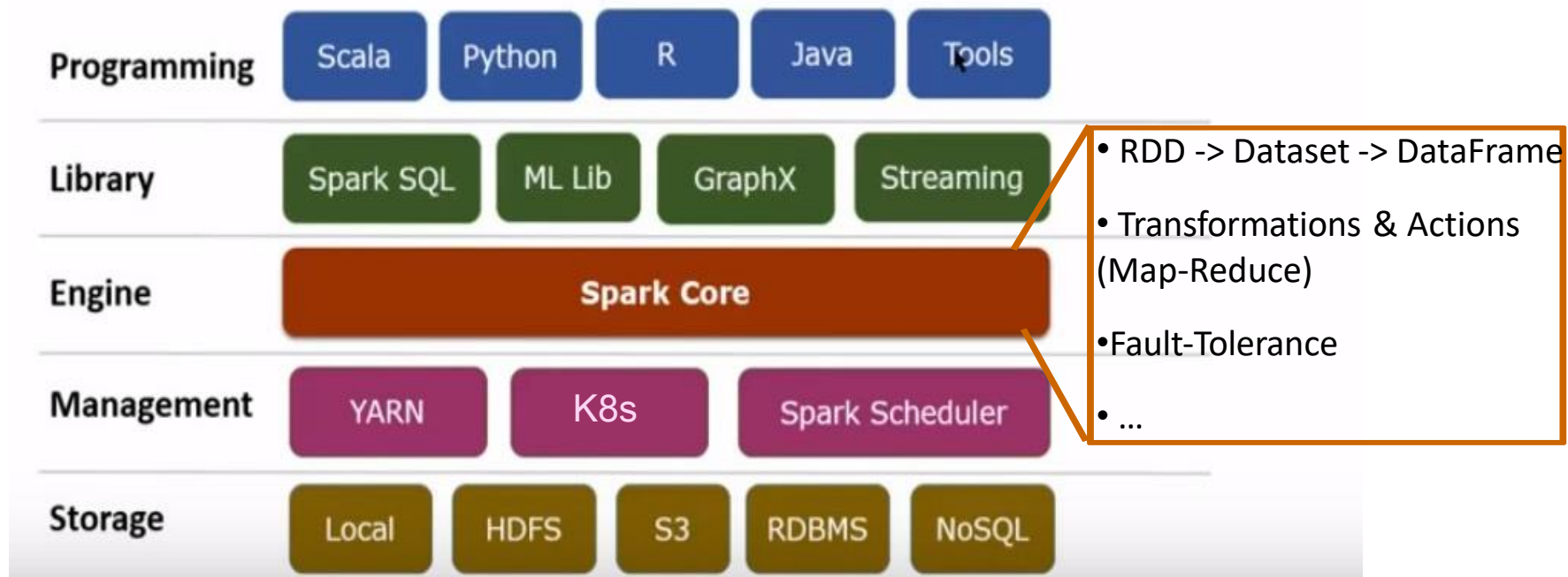
2. Spark SQL On the top of Spark, enables users to run SQL/HQL queries. Can process structured/semi-structured data and run queries up to 100 times faster on existing deployments (Hadoop for instance).

1. Spark components



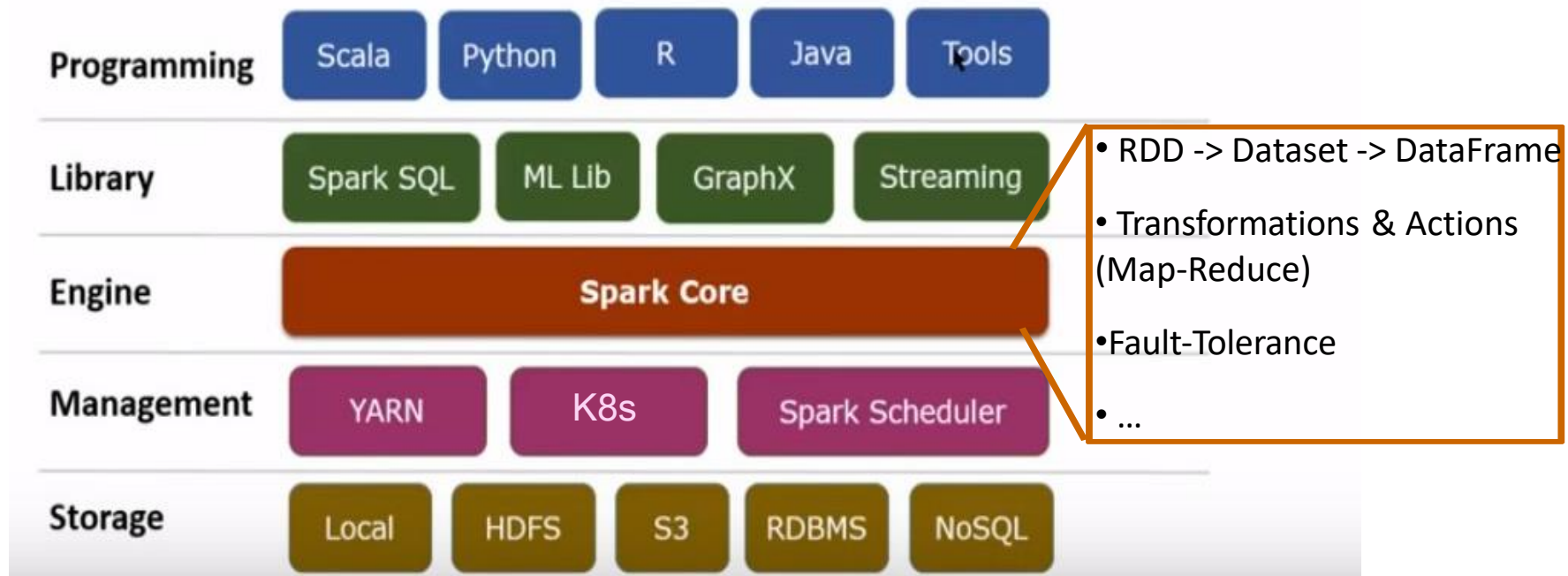
3. Spark Streaming enables a powerful interactive and data analytics applications. Live streams are converted into micro-batches and executed on top of spark core.

1. Spark components



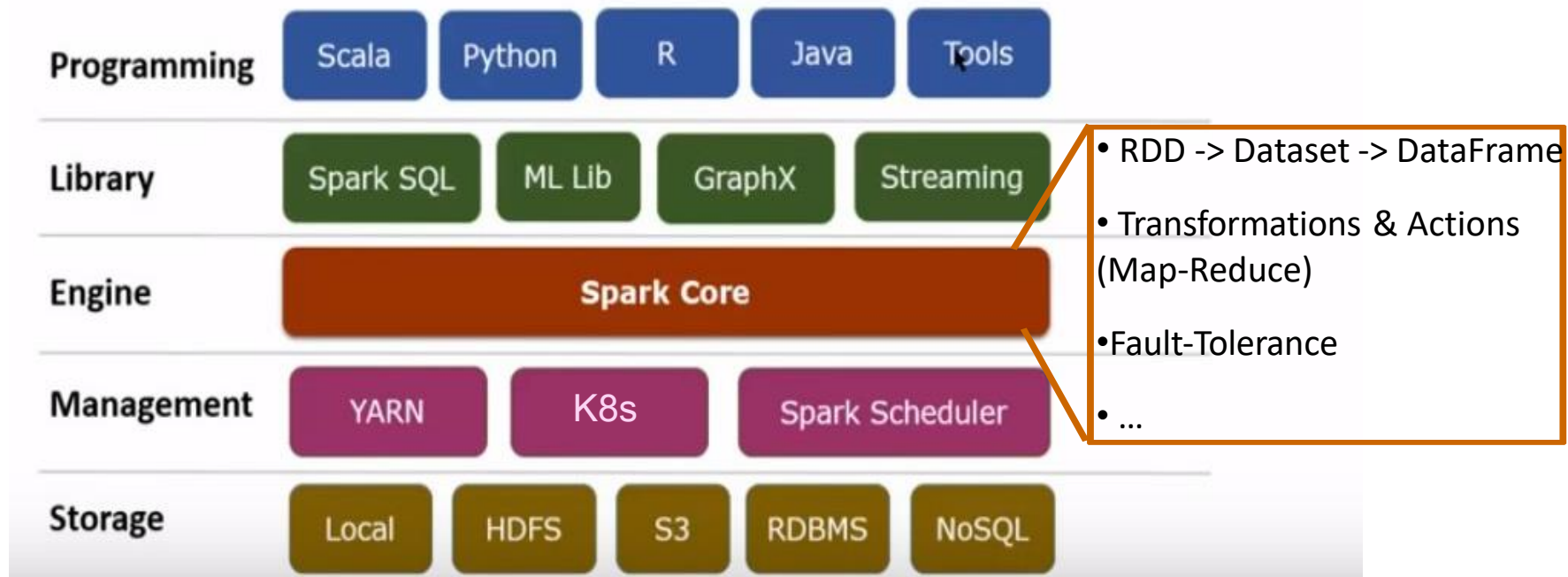
4. Spark MLlib delivers both efficiencies as well as the high-quality algorithms. Good choice for a data scientist : capable of in-memory data processing, that improves the performance of iterative algorithm drastically.

1. Spark components



5. Spark GraphX is the graph computation engine built on top of Apache Spark Core that enables to process graph data at scale.

1. Spark components



Cluster management: How to run Spark?

Local multicore: just a library in your program

EC2: scripts for launching a Spark cluster

Private cluster: Kubernetes, YARN, Standalone Mode

1. Spark components: APIs

- RDD (low-level)
- Dataset (typed)
- DataFrame (standard)

Recommended: DataFrame

1. Spark languages: Java, Scala, Python, R

Java API:

```
JavaRDD<String> lines = spark.textFile(...);  
errors = lines.filter(  
    new Function<String, Boolean>() {  
        public Boolean call(String s) {  
            return s.contains("ERROR");  
        }  
    });  
errors.count()
```

Scala (Dataset API) :

```
val lines = spark.textFile(...)  
  
errors = lines.filter(s =>  
    s.contains("ERROR"))  
// can also write  
filter(_.contains("ERROR"))  
  
errors.count
```

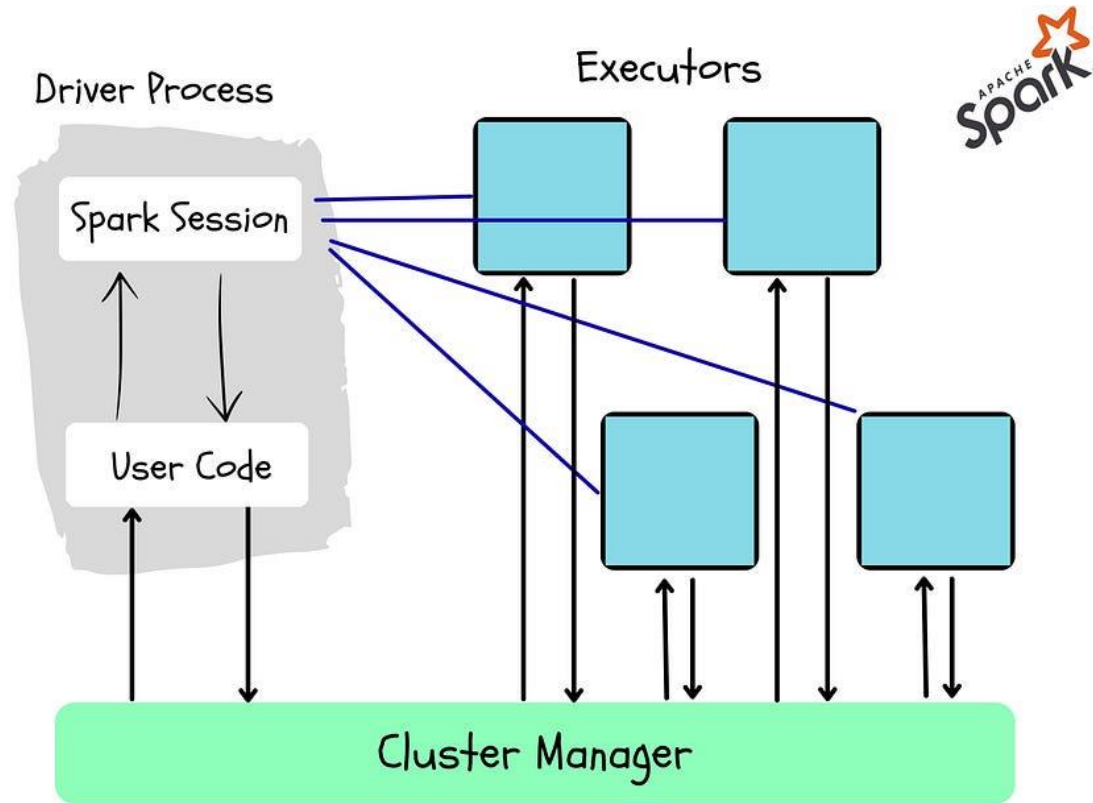
Scala (Dataframe API) :

```
val df = spark.read.text(...)  
val errors= df.filter(col("value").contains("ERROR"))  
val count = errors.count()
```

SPARK technology

1. Main objectives, features and components
- 2. Simple Spark application execution scheme**
3. RDD: Resilient Distributed Dataset
4. Detailed Spark application execution scheme
5. From RDD to Dataset and Dataframe
6. Spark Streaming

2. Simple Spark application execution plan



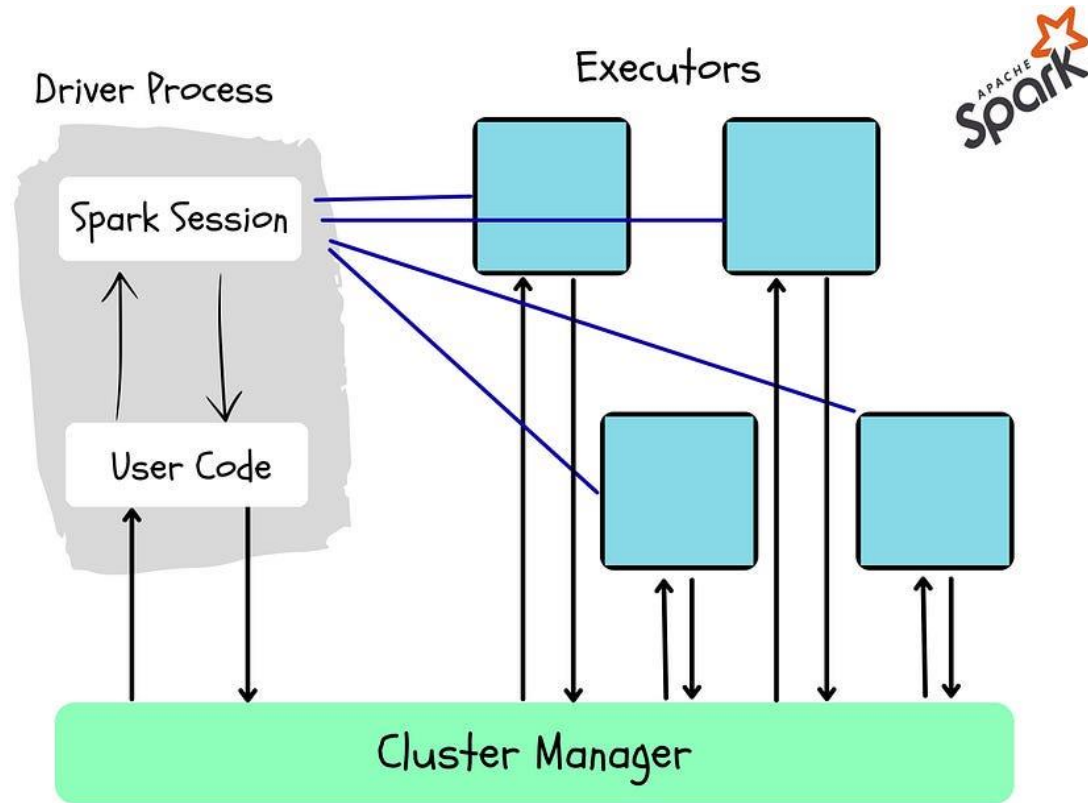
Driver Role: Acts like a manager in a distributed system.

Responsible for orchestrating the execution of a Spark application.

Takes the user's code, builds an execution plan, divides it into tasks, schedules them across executors, coordinates their execution, and collects the final results.

Unlike executors, the driver does not perform the actual data processing tasks.

2. Simple Spark application execution plan



Executor Role : Worker nodes in a Spark application responsible for executing tasks assigned by the Spark Driver.

Handle data processing in // using the CPU and memory resources allocated to them.
Can run multiple tasks simultaneously for efficient distributed computation.

SPARK technology

1. Main objectives, features and components
2. Simple Spark application execution scheme
- 3. RDD: Resilient Distributed Dataset**
4. Detailed Spark application execution scheme
5. From RDD to Dataset and Dataframe
6. Spark Streaming

3. RDD : Resilient Distributed Dataset

RDDs are the building blocks of any Spark application. RDDs Stands for:

- **Resilient:** Fault tolerant and is capable of rebuilding data on failure
- **Distributed:** Distributed data among the multiple nodes in a cluster for parallel execution
- **Dataset:** Collection of data composed of separate elements but handled as a compact unit

A RDD (*Resilient Distributed Dataset*) is:

- ✓ an **immutable** (read only) dataset
- ✓ a **partitioned** dataset
- ✓ usually stored in a distributed file system

Storage example: When stored in HDFS

- One RDD → One HDFS file
- One RDD partition block → One HDFS file block
- Each RDD partition block is replicated by HDFS

3. RDD : Resilient Distributed Dataset

RDD of Strings



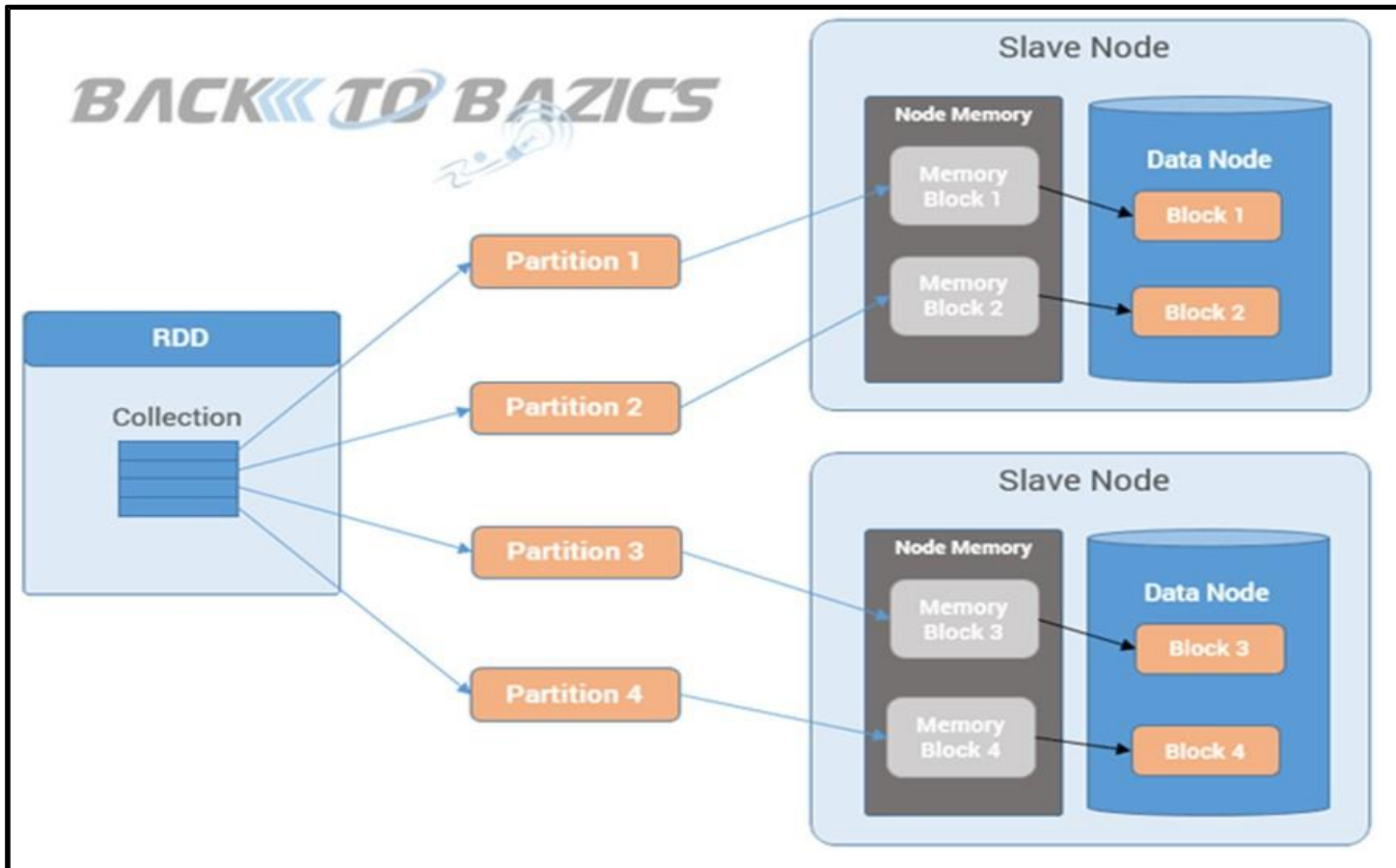
Immutable **Collection** of Objects

Partitioned and **Distributed**

Stored in **Memory**

Partitions **Recomputed on Failure**

3. RDD : Resilient Distributed Dataset



Example of a 4 partition blocks stored on 2 data nodes (no replication)

Example: 30GB uncompressed text file stored on HDFS:

With the default HDFS block size setting (128MB) $\Rightarrow$ stored in 235 blocks, which means that the RDD you read from this file would have 235 partitions.

3. RDD : Resilient Distributed Dataset

Ways to create Spark RDD : 3 ways

Parallelized collections By invoking *parallelize* method in the driver program, we can create parallelized collections.

The elements of the collection are copied to form a distributed structure that can be processed in parallel.

```
# Turn a local collection into an RDD  
sc.parallelize([1, 2, 3])
```

External datasets

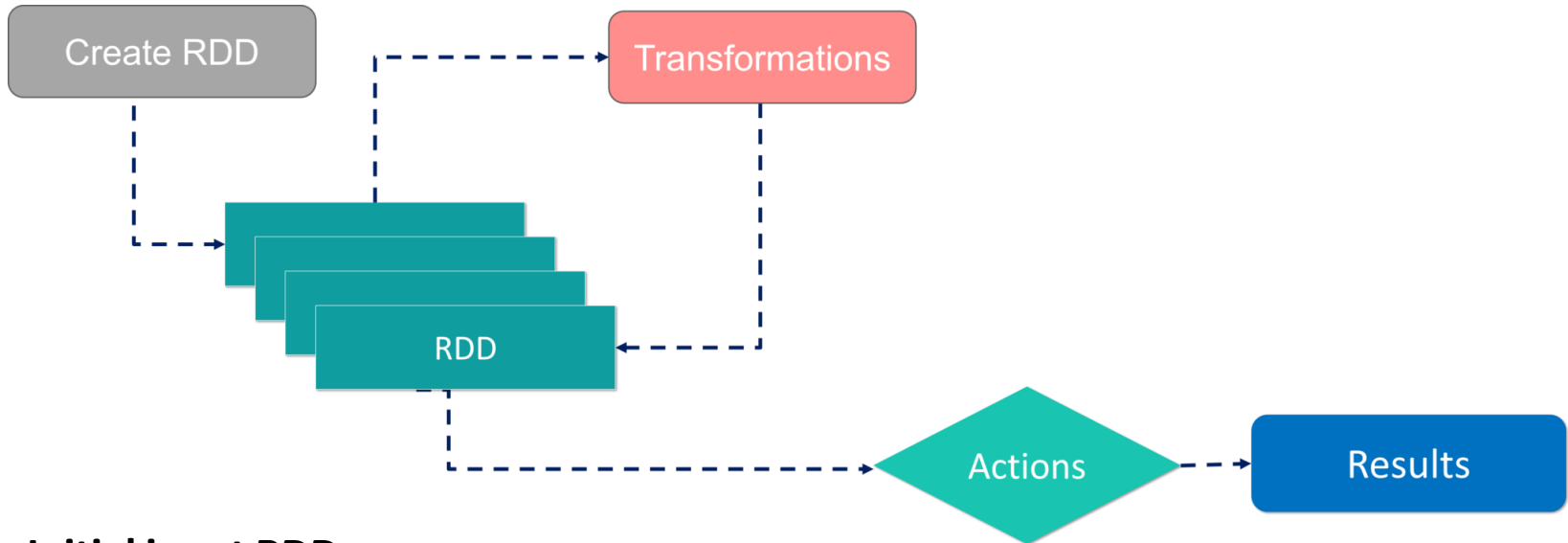
One can create Spark RDDs, by calling a *textFile* method. Hence, this method takes URL of the file and reads it as a collection of lines.

```
# Load text file from local FS, HDFS, or S3  
sc.textFile("file.txt")  
sc.textFile("directory/*.txt")  
sc.textFile("hdfs://namenode:9000/path/file")
```

Existing RDDs

Create new RDD in spark, by applying transformation operation on existing RDDs.

3. RDD : Operations



Initial input RDDs:

- are usually created from distributed files (like HDFS files),
- Spark processes read the file blocks that become in-memory RDD

Operations on RDDs:

- **Transformations** : read RDDs, compute, and generate a new RDD
- **Actions** : read RDDs and generate results out of the RDD world
- *Map* and *Reduce* are parts of the operations

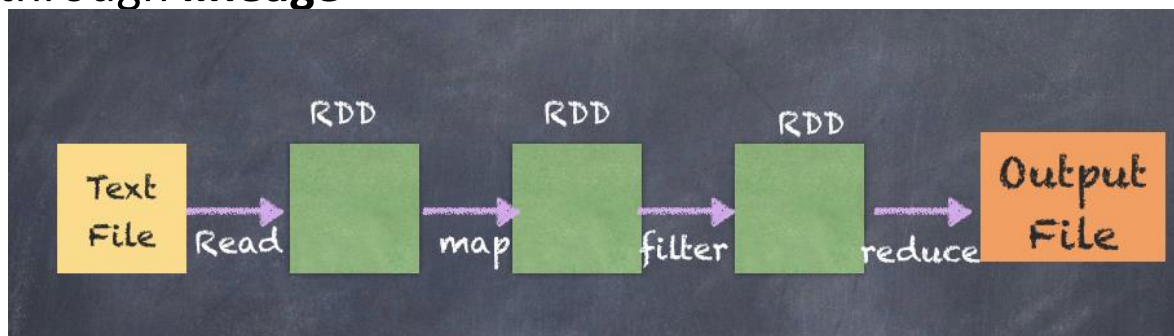
3. RDD features

- **In-memory computation**

Basically, while storing data in RDD, data is stored in memory for as long as you want to store. It improves the performance by keeping the data in memory.

- **Immutability and lineage**

Immutability means once we create an RDD, we can not manipulate it. Moreover, we can create a new RDD by performing any transformation. Also, we achieve **consistency** through **immutability**. We also achieve **Fault-tolerance** through **lineage**



3. RDD features

- **Lazy Evaluation**

Most RDD operations are lazy : Think of an RDD as a description of a series of operations.

⇒ **RDD is not data.**

```
val textFile = sc.textFile("/user/emp.txt")
```

Does nothing. It creates an RDD that says "we will need to load this file".

⇒ The file is not loaded at this point

⇒ Data inside RDDs are not evaluated on the go : Only after an action triggers all the changes or the computation is performed. Therefore, it limits how much work it has to do.

RDD operations that require observing the contents of the data cannot be lazy : **actions**.

Example :

`RDD.count` to tell you the number of lines in the file, the file needs to be read.

If you write `textFile.count`, at this point the file will be read, the lines will be counted, and the count will be returned.

Nothing is stored. An RDD is not data.

3. RDD features

- **Persistence**

In in-memory: store frequently used RDDs.

retrieve them directly from memory without going to disk.

⇒ Results in the speed of the execution + perform multiple operations on the same data.

⇒ It is only possible by storing the data explicitly in memory by calling *persist()* or *cache()* function.

Spark application developer has to add instructions to force RDD storage, and to force RDD forgetting:

```
myRDD.persist(StorageLevel) // or myRDD.cache()  
... // Transformations and Actions  
myRDD.unpersist()
```

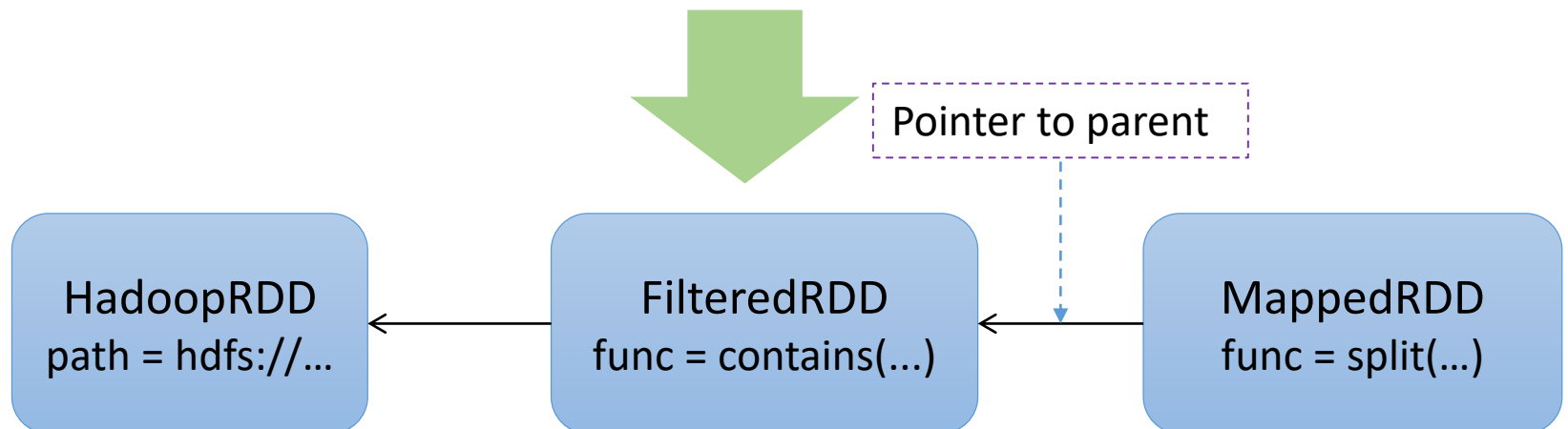
MEMORY_ONLY
MEMORY_ONLY_SER
MEMORY_AND_DISK
DISK_ONLY

3. RDD features

- **Fault-tolerance:** RDDs track the transformations used to build them (their *lineage*) to recompute lost data

If due to a worker node failure any partition of an RDD is lost, then that partition can be re-computed from the original fault-tolerant dataset using the lineage of operations.

```
messages = textFile(...).filter(lambda s: s.contains("ERROR"))  
                        .map(lambda s: s.split('\t')[2])
```



3. RDD operations

- **Basic transformations**

```
nums = sc.parallelize([1, 2, 3])
```

```
# Pass each element through a function
```

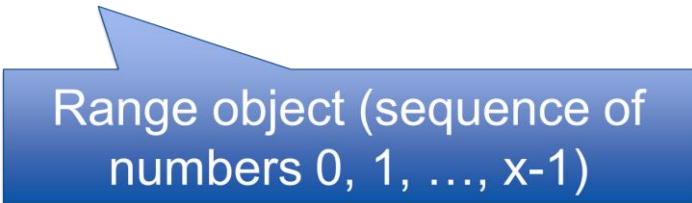
```
squares = nums.map(lambda x: x*x)    # => {1, 4, 9}
```

```
# Keep elements passing a predicate
```

```
even = squares.filter(lambda x: x % 2 == 0) # => {4}
```

```
# Map each element to zero or more others
```

```
nums.flatMap(lambda x: range(0, x))  # => {0, 0, 1, 0, 1, 2}
```



Range object (sequence of
numbers 0, 1, ..., x-1)

3. RDD operations

- **Basic actions**

```
nums = sc.parallelize([1, 2, 3])  
# Retrieve RDD contents as a local collection  
nums.collect() # => [1, 2, 3]  
# Return first K elements  
nums.take(2)    # => [1, 2]  
# Count number of elements  
nums.count()    # => 3  
# Merge elements with an associative function  
nums.reduce(lambda x, y: x + y) # => 6  
# Write elements to a text file  
nums.saveAsTextFile("hdfs://file.txt")
```

3. RDD operations

RDD operations: Working with Key-Value Pairs

Spark's “distributed reduce” transformations act on RDDs of *key-value pairs*

Python: `pair = (a, b)`

`pair[0] # => a`

`pair[1] # => b`

Scala: `val pair = (a, b)`

`pair._1 // => a`

`pair._2 // => b`

Java: `Tuple2 pair = new Tuple2(a, b); // class scala.Tuple2`

`pair._1 // => a`

`pair._2 // => b`

3. RDD operations

Some Key-Value Operations

```
pets = sc.parallelize([("cat", 1), ("dog", 1), ("cat", 2)])
pets.reduceByKey(lambda x, y: x + y)
# => {(cat, 3), (dog, 1)}

pets.groupByKey()
# => {(cat, Seq(1, 2)), (dog, Seq(1))}

pets.sortByKey()
# => {(cat, 1), (cat, 2), (dog, 1)}
```

reduceByKey also automatically implements combiners on map side

3. RDD operations

Transformation vs. Action

- A **Transformation** is a function that produces new **RDD** from the existing RDDs
- Each time a transformation is applied : a new RDD is created.

⇒ The input RDDs cannot be changed since RDD are immutable in nature.

- Transformations are lazy in nature + RDD fault-tolerance

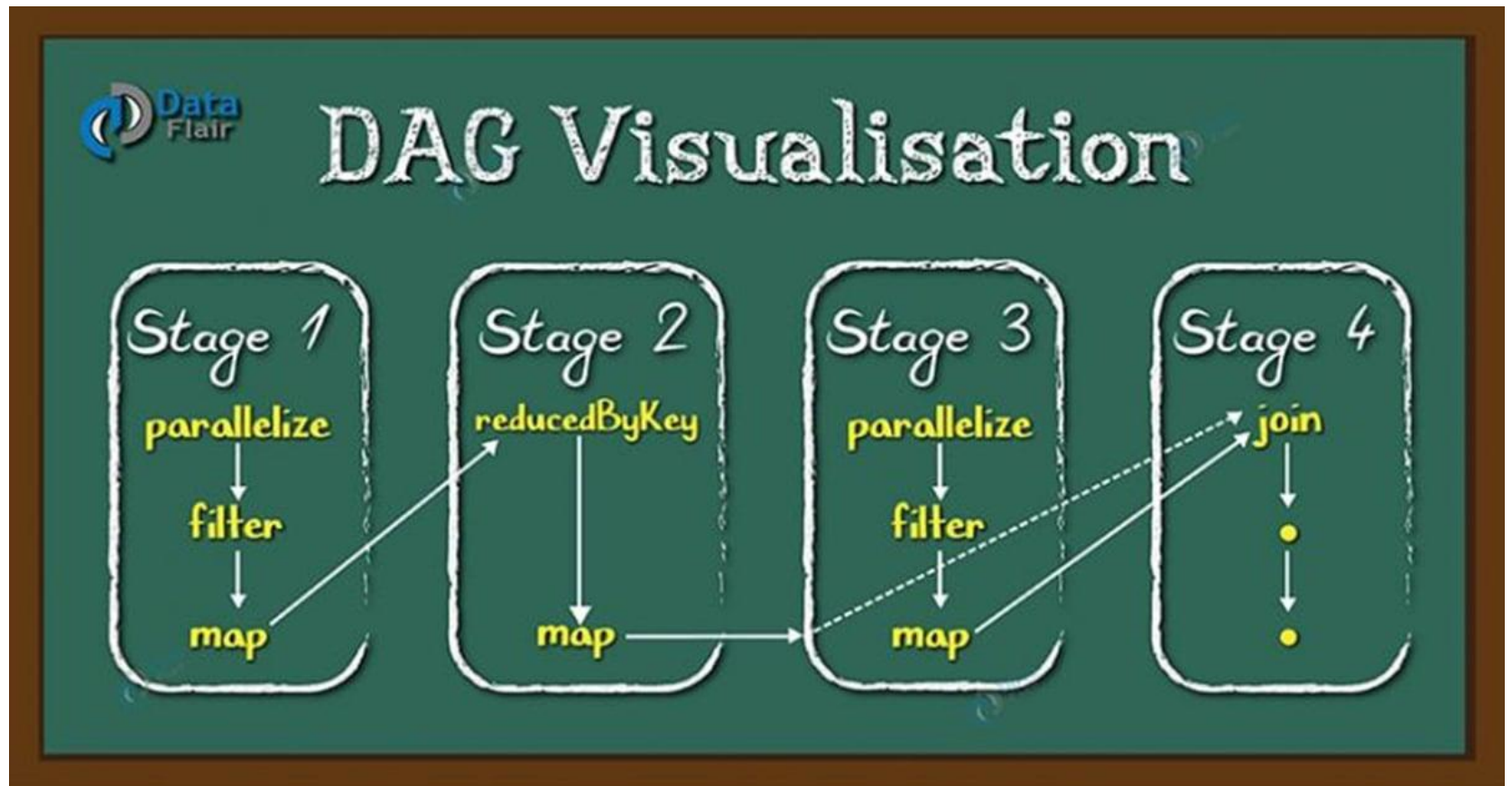
⇒ Applying transformations build an **RDD lineage**, with the entire parent RDDs of the final RDD(s).

⇒ RDD lineage, also known as **RDD operator graph** or **RDD dependency graph**. It is a logical execution plan i.e., it is a Directed Acyclic Graph (DAG) of the entire parent RDDs of RDD.

When new RDD is derived from an existing RDD using transformation: new RDD contains a pointer to the parent + Spark keeps track of all the dependencies between these RDDs using a component called the Lineage.

3. RDD operations

DAG construction from RDD operations

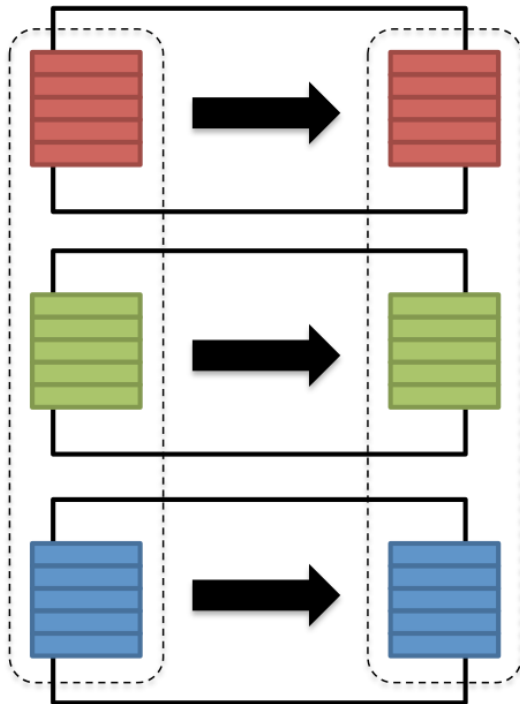


3. RDD operations

Types of Transformations

Narrow transformation

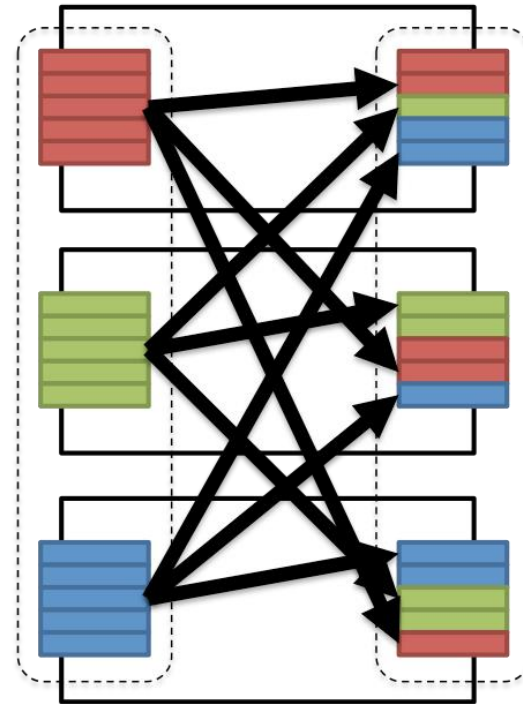
- Input and output stays in same partition
- No data movement is needed



Are the result of [map\(\)](#) and [filter\(\)](#)
`map()`, `mapPartition()`,
`flatMap()`, `filter()`, `union()`

Wide transformation

- Input from other partitions are required
- Data shuffling is needed before processing



Are the result of [groupByKey\(\)](#) and [reduceByKey\(\)](#)
`groupByKey()`, `aggregateByKey()`, `aggregate()`,
`join()`, `repartition()`

3. RDD operations

Transformations vs. Actions

- Transformations create RDDs from each other
- When we want to work with the actual dataset, at that point, **Action** is performed. When the action is triggered after the result, new RDD is not formed as for transformation.

=> In Spark, the role of an **action** is to return a value to the driver program after running a computation on the dataset.

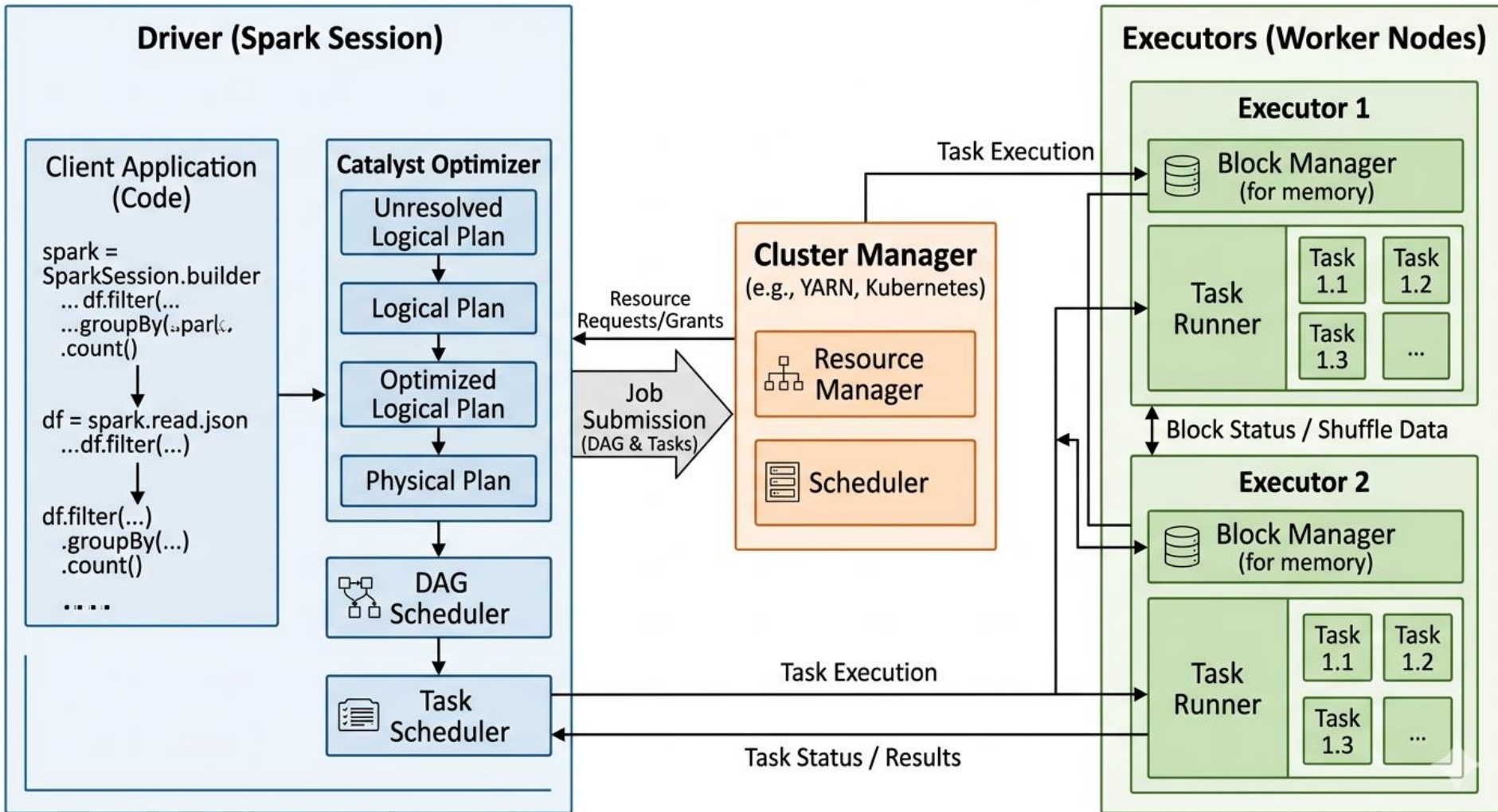
3. RDD actions: examples

Action	Description
<code>reduce(func)</code>	Aggregate the elements of the dataset using a function <code>func</code> (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
<code>collect()</code>	Returns all the elements of the dataset as an <u>array</u> at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
<code>count()</code>	It returns the number of elements in the dataset.
<code>first()</code>	It returns the first element of the dataset (similar to <code>take(1)</code>).
<code>take(n)</code>	It returns an array with the first <code>n</code> elements of the dataset.
<code>takeOrdered(n, [ordering])</code>	It returns the first <code>n</code> elements of the RDD using either their natural order or a custom comparator.
<code>saveAsTextFile(path)</code>	It is used to write the elements of the dataset as a text file (or set of text files) in a given directory in the local filesystem, HDFS or any other Hadoop-supported file system. Spark calls <code>toString</code> on each element to convert it to a line of text in the file.
<code>saveAsSequenceFile(path)</code> (Java and Scala)	It is used to write the elements of the dataset as a Hadoop SequenceFile in a given path in the local filesystem, HDFS or any other Hadoop-supported file system.
<code>saveAsObjectFile(path)</code> (Java and Scala)	It is used to write the elements of the dataset in a simple format using Java serialization, which can then be loaded using <code>SparkContext.objectFile()</code> .
<code>countByKey()</code>	It is only available on RDDs of type <code>(K, V)</code> . Thus, it returns a hashmap of <code>(K, Int)</code> pairs with the count of each key.

SPARK technology

1. Main objectives, features and components
2. Simple Spark application execution scheme
3. RDD: Resilient Distributed Dataset
- 4. Detailed Spark application execution scheme**
5. From RDD to Dataset and Dataframe
6. Spark Streaming

4. Detailed Spark application execution plan



SPARK technology

1. Main objectives, features and components
2. Simple Spark application execution scheme
3. RDD: Resilient Distributed Dataset
4. Detailed Spark application execution scheme
- 5. From RDD to Dataset and Dataframe**
6. Spark Streaming

From RDD to Dataset and Dataframe

The Shift Toward Structured APIs

The problem with RDDs:

Spark does not know the data structure inside an RDD (“black box”) : no optimization
⇒ The DataFrame Solution: An RDD with a schema (columns/types).

1. DataFrames:

- Higher-level abstraction, introduced in Spark 1.3, designed for structured data processing.
- Data in a tabular format (similar to pandas DataFrame or SQL tables).
- + Schema-based — each column has a name and type.
- + Optimized using the Catalyst optimizer.
- + Provides a declarative API for data manipulation.
- Less control over data transformations compared to RDDs
- Schema enforcement might make it less flexible for some use cases.

From RDD to Dataset and Dataframe

2. *Datasets*:

- Introduced in Spark 1.6 as an extension of DataFrames.
- Combine the performance optimization of DataFrames with the type safety and object-oriented programming benefits of RDDs.

Characteristics of Datasets:

- Schema-based, with compile-time type safety (in languages like Scala and Java).
- Supports functional transformations similar to RDDs.
- Provides better error handling due to type safety.

+ Type safety ensures fewer runtime errors.

+ Combines the best of both worlds (RDDs and DataFrames).

+ Optimized using Catalyst.

- Full benefits are available only in strongly typed languages like Scala and Java.
- Slightly more verbose than DataFrames.

Starting with Spark version 2, the Dataset and DataFrame APIs have been unified.

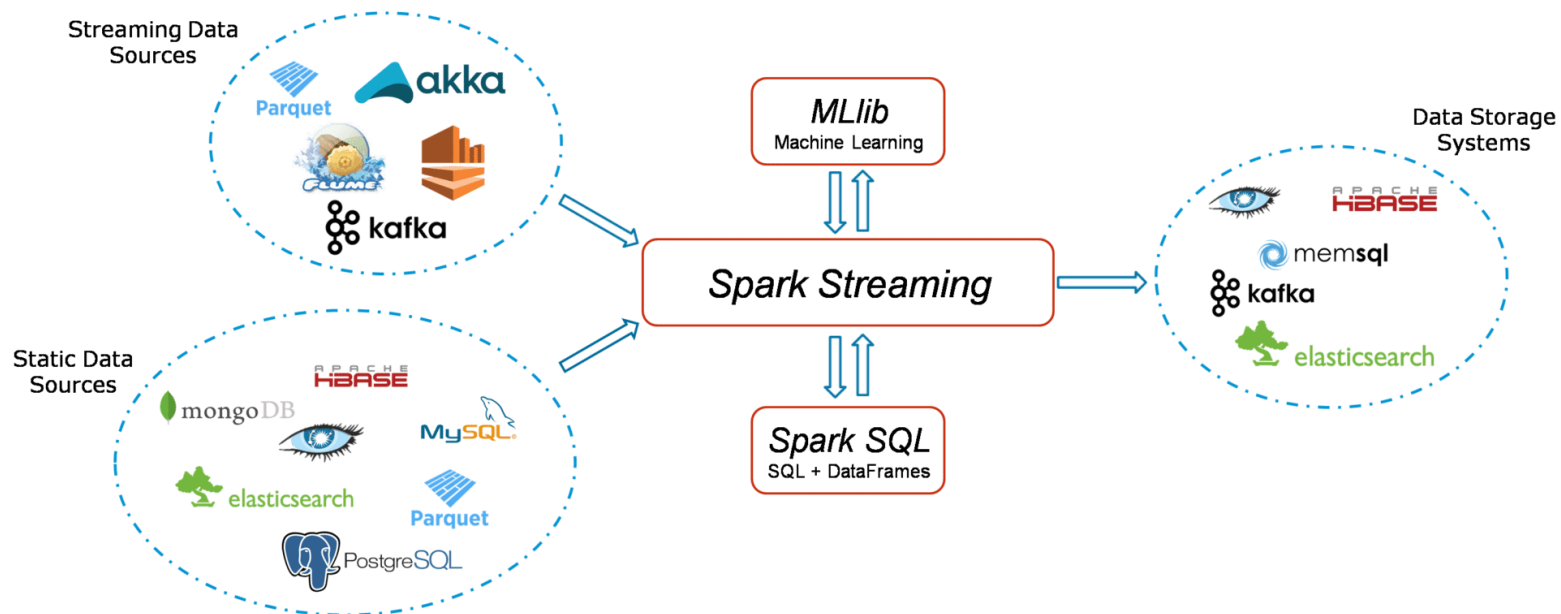
RDD vs. Dataset vs. DataFrame

Critère	RDD	Dataset	DataFrame
Level	Low	Med.	High
Typing	No	Yes	No (schema)
Optimisation	✗	⚠ partial (Catalyst)	✓ Complete (Tungsten)
Performance	Average	Good	Excellent
Current usage	Rare	Limited	Standard

SPARK technology

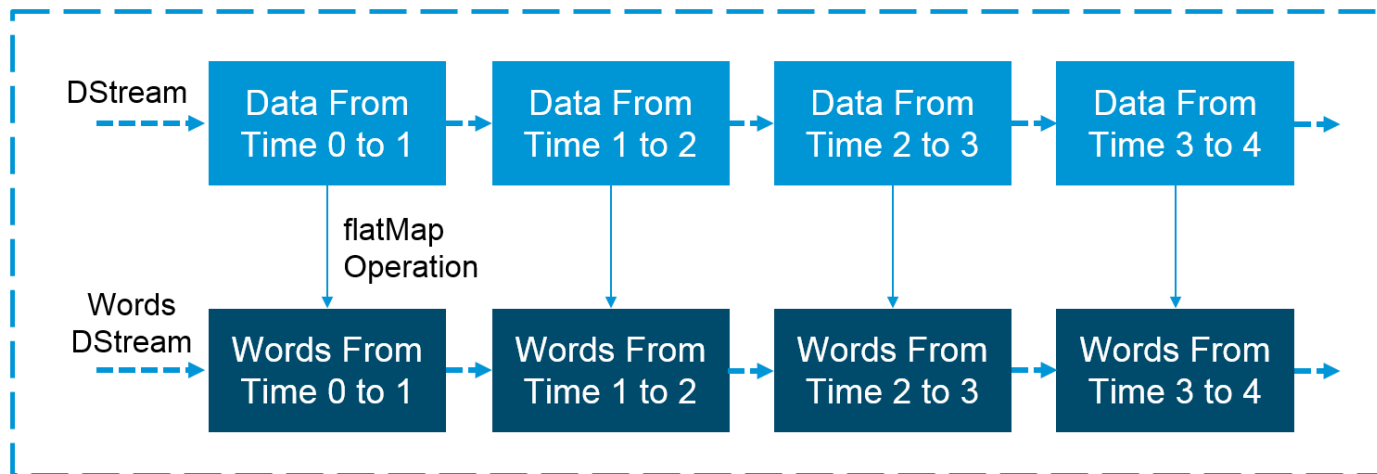
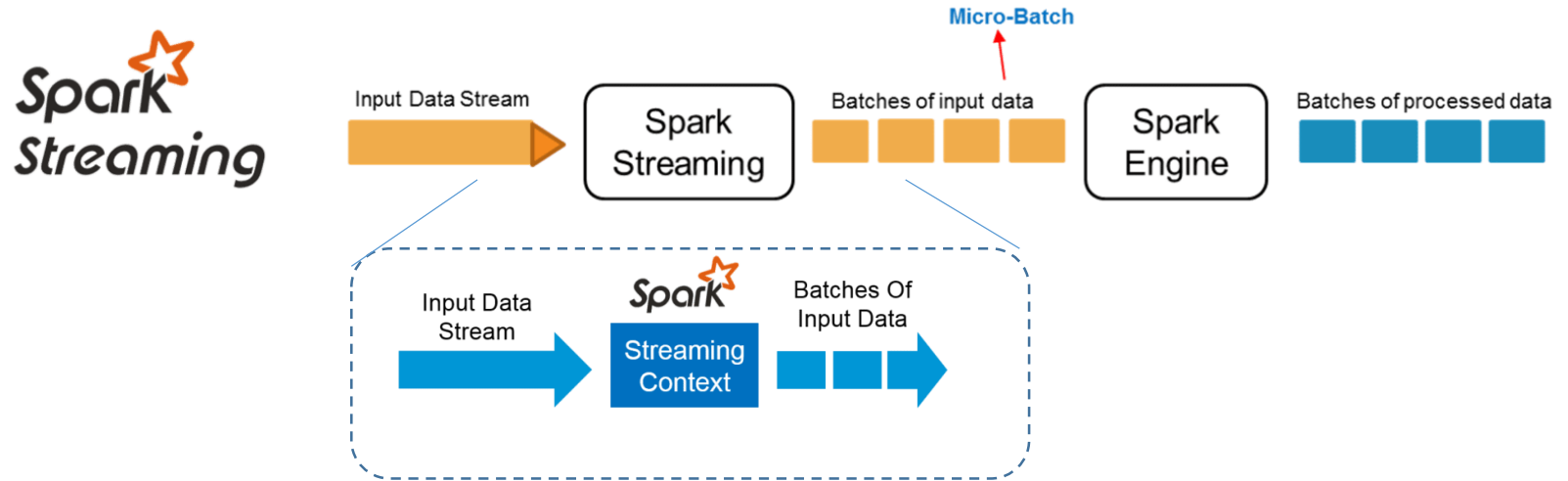
1. Main objectives, features and components
2. Simple Spark application execution scheme
3. RDD: Resilient Distributed Dataset
4. Detailed Spark application execution scheme
5. From RDD to Dataset and Dataframe
- 6. Spark Streaming**

6. Spark Streaming



- Extension of the core Spark API
- Enables scalable, high-throughput, fault-tolerant stream processing of live data streams.
- Used to stream live data and processing can happen in real time.

6. Spark Streaming

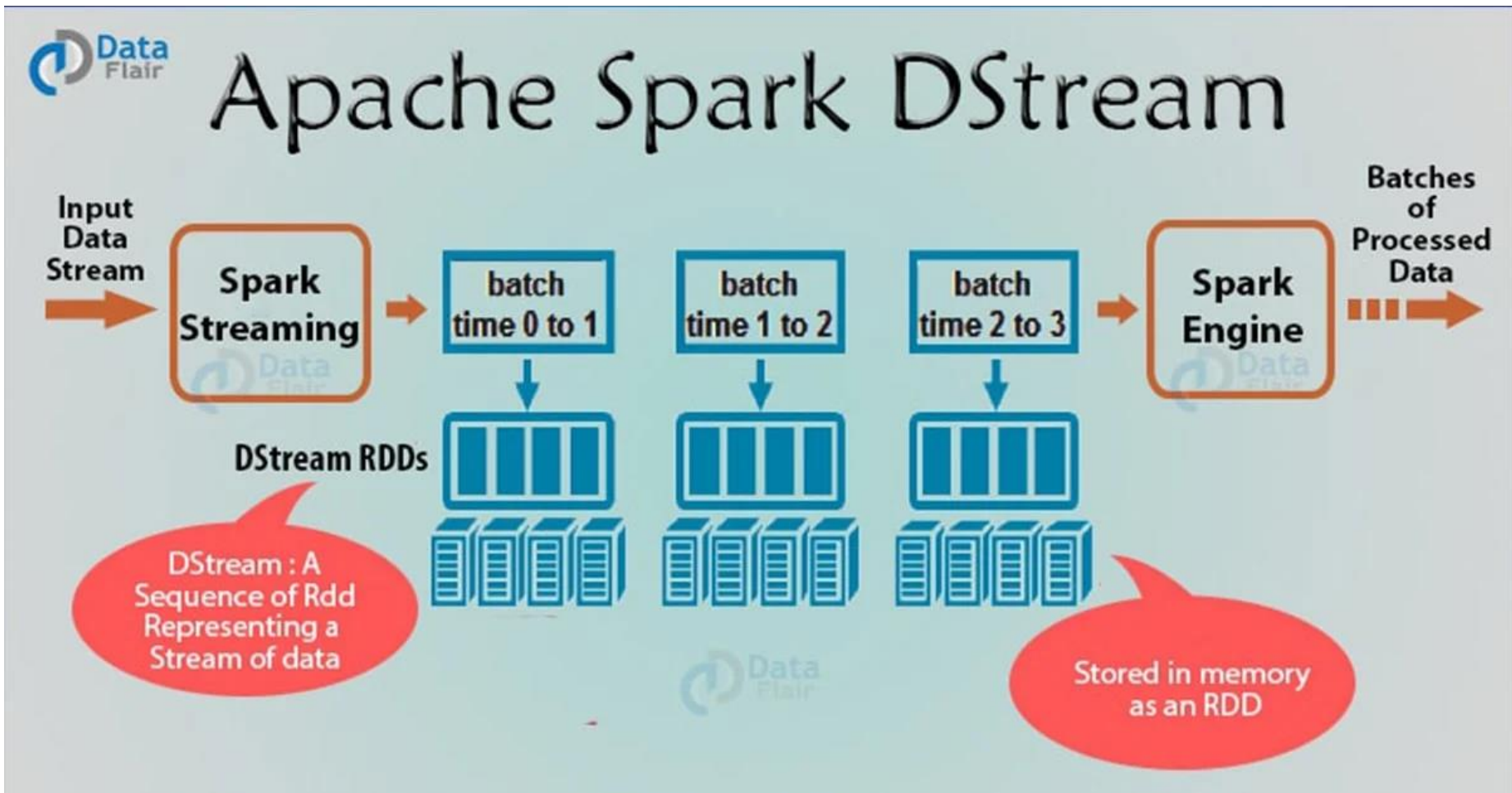


Extracting words from an Input DStream

6. Spark Streaming

- Spark Streaming inputs sources : **Flume, HDFS, and Kafka** etc. Sends the processed data to filesystems, database or live dashboards.
- Input data stream: divided into micro-batches of data and then generates the final stream of the result in batches.
- Apache Spark *DStream* (Discretized Streams): ***DStream is a continuous stream of RDDs (Spark abstraction)***
- Every RDD in *DStream* contains data from a certain interval
- Any operation on a *DStream* applies to all the underlying RDDs.
- Like RDD, Spark *DStream* also support two types of Operations: Transformations and output Operations (save, print etc..)

6. Spark Streaming



6. Spark Streaming

Transformations on DStreams:

- Any operation applied on a DStream translates to operations on the underlying RDDs.
- Transformations allow the data from the input DStream to be modified similar to RDDs.
- DStreams support many of the transformations available on normal Spark RDDs.

Output operations on DStreams:

- Allow DStream's data to be pushed out to external systems like databases or file systems. Output operations trigger the actual execution of all the DStream transformations.

